SuperStack — Complete Command List

Every command, from SuperStack itself to all 10 integrated repos, one line each.

SuperStack CLI (run in your terminal)

Command	Description
`superstack install`	Install/verify all 10 repos plus the global router and hooks
`superstack install --with-design`	Same as above, and also installs open-design
`superstack init <name>`	Create a brand new SuperStack-enabled project
`superstack migrate <path>`	Add SuperStack (vault, graph, config) to an existing project
`superstack doctor`	Health-check every part of the stack and suggest fixes
`superstack update`	Pull the latest version of all 10 repos and the router itself
`superstack uninstall`	Remove SuperStack's global files (leaves individual plugins installed)
`superstack version`	Print the installed SuperStack version

Real example: You cloned a two-year-old project and want it wired in.

```
cd old-project
superstack migrate .
```

This builds a fresh code graph and vault for it without touching your code.

SuperStack In-Chat Commands (type inside Claude Code)

Command	Description
`/skip`	Skip the preview for this one prompt (not for heavy/destructive tasks)
`/ponytail lite`	Set code style to more exploratory, less minimal
`/ponytail full`	Set code style to balanced (default)
`/ponytail ultra`	Set code style to maximum minimalism
`/ponytail auto`	Let Claude choose the level automatically again
`/superstack-status`	Show which tools are active and whether memory/graph are healthy
`/save`	Save the current conversation into the project's vault
`/save <name>`	Save the conversation under a specific name

Real example: You're deep in a large cleanup touching 15 files and don't want to be asked every time — but since it's over the 10-file threshold, the preview will show anyway even with `/skip`. That's intentional: `/skip` only waives the ask for small, normal tasks.

---

1. ECC — Quality, Security & Optimization

Command	Description
`/ecc:plan`	Plan a feature or task implementation
`/ecc:code-review`	Review code for quality and security issues
`/ecc:build-fix`	Diagnose and fix build/compile errors
`/ecc:refactor-clean`	Remove dead code and simplify structure
`/ecc:quality-gate`	Run the full verification checklist before shipping
`/ecc:learn`	Extract reusable patterns from this session
`/ecc:checkpoint`	Save the current verification state
`/ecc:security-scan`	Run a security vulnerability scan

`/ecc:update-docs`	Regenerate documentation from current code
`/ecc:update-codemaps`	Refresh the internal map of the codebase
`/ecc:test-coverage`	Analyze how much of the code is tested
`/ecc:go-review`	Review Go code specifically
`/ecc:go-test`	Run Go-specific test-driven workflow
`/ecc:go-build`	Fix Go build errors
`/ecc:python-review`	Review Python code against PEP 8
`/ecc:setup-pm`	Auto-detect and configure the package manager
`/ecc:skill-create`	Generate a new skill from git history patterns
`/ecc:instinct-status`	View instincts (learned patterns) so far
`/ecc:instinct-import`	Import instincts from a file
`/ecc:instinct-export`	Export your learned instincts
`/ecc:evolve`	Cluster instincts into a reusable skill
`/ecc:prune`	Delete expired/unused pending instincts
`/ecc:pm2`	Manage background services via PM2
`/ecc:multi-plan`	Split a big task into parallel planning sub-agents
`/ecc:multi-execute`	Run a multi-agent orchestrated workflow
`/ecc:multi-backend`	Coordinate multiple backend services at once
`/ecc:multi-frontend`	Coordinate multiple frontend services at once
`/ecc:multi-workflow`	General-purpose multi-service orchestration
`/ecc:sessions`	View past session history
`/ecc:harness-audit`	Audit the reliability of your current agent setup
`/ecc:loop-start`	Start a controlled autonomous work loop
`/ecc:loop-status`	Check the status of an active loop

`/ecc:model-route`	Route sub-tasks to different models by complexity
`/ecc:learn-eval`	Extract and score patterns from a session
`/ecc:promote`	Promote a project-specific instinct to global
`/ecc:projects`	List all tracked projects and their instinct stats
`npx ecc consult "<topic>"`	Ask ECC's knowledge base a direct question
`npx ecc install --profile <name>`	Install ECC with a specific feature profile
`ecc work-items sync-github`	Sync tracked work items with a GitHub repo
`node scripts/ecc.js doctor`	Check ECC's own installation health
`npx ecc-agentshield scan`	Scan for security vulnerabilities
`npx ecc-agentshield scan --fix`	Scan and automatically fix what it can

Real example (complex one): You inherited a legacy Go microservice with no tests and unclear ownership.

```
/ecc:go-review
```

Then:

```
/ecc:go-test
```

This reviews the Go-specific patterns first, then sets up a proper test-driven workflow instead of you guessing where to start.

---

2. Superpowers — Development Workflow

Command	Description
`/brainstorm`	Explore ideas freely, without writing any code yet
`/write-plan`	Turn an idea into a concrete, detailed implementation plan

`/execute-plan`	Carry out a written plan, running independent parts in parallel
`/think-pair-share`	Work through a decision collaboratively, weighing trade-offs
`/preserving-productive-tensions`	Keep multiple valid solutions open instead of collapsing too early

Real example: You're not sure if you want a monolith or microservices.

```
/brainstorm "monolith vs microservices for this project"
```

Claude explores both directions without committing code, so you can decide with a clear picture instead of Claude jumping straight to implementation.

---

3. GSD Core — Long-Feature Context Management

Command	Description
`/gsd-new-project`	Initialize a brand-new project under GSD's structure
`/gsd-onboard`	Bring an existing codebase into GSD's phase system
`/gsd-sync`	Sync GSD's tracked state with the current project
`/gsd-status`	Check what phase the current feature is in
`/gsd-plan-feature`	Plan a new feature across discuss/plan/execute/test/ship phases
`/gsd-plan-bugfix`	Plan a targeted bug fix the same way
`/gsd-estimate`	Estimate effort for planned work
`/gsd-review`	Review completed work against the plan
`/gsd-ready-to-ship`	Run final checks before releasing

Real example: A feature is big enough to span several days and multiple files, and you're worried Claude will "forget" earlier decisions halfway through.

```
/gsd-plan-feature "add multi-tenant support"
```

GSD breaks it into phases with fresh context at each step, so nothing gets lost partway through a long build.

---

4. Ponytail — Code Minimalism

Command	Description
`/ponytail lite`	Minimal-intensity suggestions — leaves room to explore
`/ponytail full`	Default balance of minimalism and flexibility
`/ponytail ultra`	Forces the shortest, most minimal solution possible
`/ponytail off`	Turn off lazy-dev mode entirely
`/ponytail-review`	Review existing code against minimalism principles
`/ponytail-audit`	Audit the whole codebase for over-engineering
`/ponytail-debt`	Track shortcuts taken and technical debt incurred
`/ponytail-gain`	See how much simpler the code became with ponytail active
`/ponytail-help`	Show a quick reference of ponytail commands

Real example: You suspect your codebase has grown bloated over time.

```
/ponytail-audit
```

It flags over-engineered spots — unnecessary abstractions, unused flexibility — so you know exactly what to simplify first.

---

5. Graphify — Code Knowledge Graph

Command	Description
`/graphify`	Map the entire current project into a knowledge graph
`graphify install`	Register graphify with your AI assistant

`graphify query "<question>"`	Ask a question about how the codebase is structured
`graphify community-detect`	Automatically find and cluster related subsystems
`graphify verify <change>`	Formally verify that a code change is safe (enterprise)
`graphify diff`	Show what changed in the graph since last run
`graphify export <format>`	Export the graph as JSON, GraphML, HTML, CSV, or Markdown
`graphify label`	Auto-name the detected clusters/communities
`graphify doctor`	Check graphify's own installation health

Real example: You just joined a project with 80,000 lines of unfamiliar code.

```
graphify .
```

Then open `graph.html` in your browser — you get a visual map showing which files are central "hubs" and how everything connects, instead of reading every file one by one.

---

6. claude-obsidian — Second Brain / Vault

Command	Description
`/wiki`	Check vault setup, or continue where you left off
`ingest [file]`	Read a source file and file it into 8-15 linked wiki pages
`ingest all of these`	Batch-process several sources and cross-reference them
`what do you know about X?`	Get a cited answer synthesized from the vault
`/save`	File the current conversation as a wiki note
`/save [name]`	Save with a specific title
`/autoresearch [topic]`	Autonomously research a topic online and file the findings
`/canvas`	Open or create a visual canvas of notes/images/PDFs

`/think [problem]`	Apply a structured 10-principle thinking process to a problem
`lint the wiki`	Check the vault for orphaned notes, dead links, and gaps
`update hot cache`	Refresh the short-term memory summary file

Real example: You just finished a long design discussion in chat and want it kept for later.

```
/save "auth architecture decision"
```

It's now a permanent, searchable note — next time you ask "why did we choose JWT over sessions," Claude can answer from this saved page.

---

7. obsidian-skills — Vault File Literacy

Command	Description
`/plugin install obsidian@obsidian-skills`	Install the skill that teaches proper vault file handling

(This repo has no commands of its own — it silently improves how correctly Claude reads and writes Markdown, Bases, and Canvas files inside your vault.)

---

8. andrej-karpathy-skills — Coding Discipline

Command	Description
`/plugin install andrej-karpathy-skills@multica-ai`	Install the always-on coding discipline guideline

(No further commands — once installed, it silently applies on every prompt: no silent assumptions, no over-engineering, goal-driven execution.)

---

9. ui-ux-pro-max-skill — Design Intelligence

Command	Description
`uipro init --ai <platform>`	Install the design database skill for your AI platform
`uipro versions`	List available versions of the design skill
`uipro update`	Refresh the design database to the latest
`uipro uninstall`	Remove the design skill
`python3 search.py "<query>" --domain <domain>`	Search the design database directly

Real example: You need a color palette and font pairing for a finance dashboard, not just "something that looks nice."

```
uipro search "finance dashboard" --domain color-palettes
```

Returns specific, tested palette options suited to that exact product type, instead of a generic guess.

---

10. open-design — Design Artifact Generation

Command	Description
`od plugin list`	List all installed design plugins
`od plugin search "<query>"`	Search available design plugins by keyword
`od plugin info <plugin-id>`	See what a plugin needs as input and what it outputs
`od plugin install <plugin-id>`	Install a plugin from the registry
`od plugin apply <plugin-id> --input key=value`	Run a plugin to generate an actual design artifact
`od plugin upgrade <plugin-id>`	Update an installed plugin
`od plugin uninstall <plugin-id>`	Remove an installed plugin

Real example: You need an actual slide deck file, not just design advice.

```
od plugin search "pitch deck"
od plugin apply pitch-deck-generator --input topic="Q3 results"
```

This produces a real, exportable `.pptx` file — not just a description of what the deck should look like.

Grand Total

Category	Count
SuperStack CLI	8
SuperStack in-chat	8
ECC	40+
Superpowers	5
GSD Core	9
Ponytail	9
Graphify	9
claude-obsidian	11
obsidian-skills	1 (install only)
andrej-karpathy-skills	1 (install only)
ui-ux-pro-max-skill	5
open-design	7
TOTAL	**~113+ commands**

Reminder: SuperStack's router auto-triggers the most common ~30% of these for you based on what you type. Every command above still works if you type it directly, any time.